Free products and weak products

Simon Frost

Table of contents

1 Overview 1

2 Two summaries of one record 1

3 The free product 2

4 The weak pair 3

5 Not unique 3

1 Overview

Cellular automata over one group have products: a pair of automata into two full shifts is one automaton into their product. Generalised cellular automata between different groups have only a *weak* product. It lives on the free product of the two groups, and pairs exist but are not unique (Castillo-Ramirez et al. 2025). This vignette builds free products, pairs two generalised automata, and shows a second pair that the projections cannot tell apart from the first.

```
using CategoricalCellularAutomata
```

2 Two summaries of one record

A pest trap is read every month, and the record of a year is a configuration on the twelve months, the cyclic group `Z_12`. Two reports summarise it. A quarterly report says whether the pest was caught in a quarter. A report every four months gives the total caught in a four-month period, capped at 3. Quarter `q` starts in month `3q`, and period `r` starts in month `4r`. These are homomorphisms from `Z_4` and `Z_3` into `Z_12`, and each report is a generalised cellular automaton over its homomorphism:

```
months = CyclicGroup(12)
counts = FinAlphabet(0:3)
quarter_start = GroupHom(q -> 3q, CyclicGroup(4), months)
period_start = GroupHom(r -> 4r, CyclicGroup(3), months)
quarterly = GeneralizedCA(quarter_start, [0, 1, 2], p -> Int(any(>(0), p));
dom=counts,
                          codom=FinAlphabet(0:1))
four_monthly = GeneralizedCA(period_start, [0, 1, 2, 3], p -> min(3, sum(p));
dom=counts)
record = Configuration(months, [0, 0, 1, 2, 0, 0, 0, 3, 1, 0, 0, 0])
println("quarterly:    ", parent(quarterly(record)))
println("four-monthly: ", parent(four_monthly(record)))
```

```
quarterly:    [1, 1, 1, 0]
four-monthly: [3, 3, 1]
```

3 The free product

The two reports have the same domain but live on different groups. Their pair must live on a group that contains both `Z_4` and `Z_3` with nothing imposed between them: the free product `Z_4 * Z_3`. Its elements are words that alternate between the two groups. A word reads as "some quarters, then some four-month periods, then some quarters, ...", and multiplying words concatenates them and cancels where the same group meets itself:

```
F = FreeProduct(CyclicGroup(4), CyclicGroup(3))
w = mul(F, inclusion(F, 1)(1), inclusion(F, 2)(2))
(w, mul(F, w, inverse(F, w)), mul(F, ((1, 3),), ((1, 2),)))
```

```
(((1, 1), (2, 2)), (), ((1, 1),))
```

The free product is infinite. The number of words grows exponentially with their length:

```
[R ⇒ length(ball(F, R)) for R in 0:6]
```

```
7-element Vector{Pair{Int64, Int64}}:
 0 => 1
 1 => 6
 2 => 18
 3 => 48
 4 => 120
 5 => 300
 6 => 732
```

A homomorphism out of a free product is fixed by what it does on each factor. Here a word goes to the month reached by moving through its quarters and periods in turn:

```
to_month = induced_hom(F, quarter_start, period_start)
to_month(w)
```

```
11
```

4 The weak pair

`weak_pair` pairs the two reports into one generalised automaton, from the monthly record to the full shift of pairs over the free product, along `to_month`. Composed with the projections it gives back each report:

```
both = weak_pair(quarterly, four_monthly)
π1, π2 = weak_projections(codom(quarterly), codom(four_monthly))
(compose(both, π1) == quarterly, compose(both, π2) == four_monthly)
```

```
(true, true)
```

At a word, the pair reports both summaries for the month the word reaches:

```
[(word, to_month(word), both(record, word)) for word in ball(F, 2)[1:8]]
```

```
8-element Vector{Tuple{Tuple{Vararg{Tuple{Int64, Int64}}}, Int64, Tuple{Int64,
Int64}}}:
 ((), 0, (1, 3))
 (((1, 1),), 3, (1, 2))
 (((1, 2),), 6, (1, 3))
 (((1, 3),), 9, (0, 0))
 (((2, 1),), 4, (0, 3))
 (((2, 2),), 8, (1, 1))
 (((1, 1), (2, 1)), 7, (1, 3))
 (((1, 1), (2, 2)), 11, (0, 1))
```

5 Not unique

In a product, the pair is the *only* automaton that projects onto the two given ones. Here it is not. Suppose the quarterly report is replaced by a constant, a report that always says “caught”. A constant is a φ-automaton for every homomorphism φ, so the pair can be built along a second homomorphism that sends every quarter to month 0:

```
always = GeneralizedCA(quarter_start, [0], p -> 1; dom=counts,
codom=FinAlphabet(0:1))
stuck = GroupHom(q -> 0, CyclicGroup(4), months)
always_stuck = GeneralizedCA(stuck, [0], p -> 1; dom=counts, codom=FinAlphabet(0:1))
u = weak_pair(always, four_monthly)
```

```
v = weak_pair(always_stuck, four_monthly)
(compose(u, π1) == compose(v, π1) == always, compose(u, π2) == compose(v, π2) ==
four_monthly)
```

```
(true, true)
```

The projections cannot tell `u` and `v` apart, yet they are different maps. At a word that moves by two quarters and then a four-month period, `u` reads the record from the month that word reaches, while `v`, which ignores the quarters, reads it from month 4:

```
word = mul(F, inclusion(F, 1)(2), inclusion(F, 2)(1))
stuck_month = induced_hom(F, stuck, period_start)
(months=(to_month(word), stuck_month(word)), u=u(record, word), v=v(record, word))
```

```
(months = (10, 4), u = (1, 0), v = (1, 3))
```

So the full shift over the free product, with its projections, is a weak product: pairs exist, but they need not be unique. Whether the category of generalised cellular automata has true products is an open question (Castillo-Ramirez et al. 2025).

Castillo-Ramirez, Alonso, Alejandro Vazquez-Aceves, and Angel Zaldivar-Corichi. 2025. *Categorical Products of Cellular Automata*. https://arxiv.org/abs/2503.21567.